Complete Curriculum Plan for Developing Deployment ready Applied AI Engineers

---

Session Structure (Every Week)

Every 1.5-hour session follows this format without exception:

- 0:00 – 0:10 Quick recap + last week's assignment review (what worked, what broke)
- 0:10 – 0:55 Core concept teaching with live coding
- 0:55 – 1:10 Guided exercise (students code alongside mentor)
- 1:10 – 1:30 Assignment briefing + Q&A

Weekly rhythm:

- **Saturday session:** New concept introduced, live coding walkthrough
- **Sunday session:** Deeper dive, edge cases, students attempt problems, you debug together
- **Monday–Friday:** Self-paced assignment work, GitHub submissions by Friday night

---

Phase 0 — Engineering Foundations

Duration: 2 months

Month 1 — Python Engineering & Developer Environment

Week 1 — Developer Environment & Professional Python Setup

Saturday session:

- Why terminal matters — GUI is for users, terminal is for engineers

- Installing WSL (Windows Subsystem for Linux) on Windows, or native Ubuntu setup
- Terminal navigation: `ls`, `cd`, `mkdir`, `cp`, `mv`, `rm`, `cat`, `grep`
- Installing Python properly (not from Microsoft Store — from python.org or `pyenv`)
- VS Code setup: extensions (Pylance, Black formatter, GitLens, Python), settings.json basics

Sunday session:

- Virtual environments: why they exist, `venv` creation, activation, deactivation
- `pip` properly: installing packages, `requirements.txt`, `pip freeze`
- Running Python from terminal, not just pressing a play button
- File structure of a real Python project (not everything in one file)

Assignment: Set up complete dev environment, create a virtual environment, install 3 packages, write a `requirements.txt`, submit the folder structure screenshot + requirements file on GitHub

---

Week 2 — Python Beyond Basics: Writing Code That Doesn't Break

Saturday session:

- Type hints — what they are and why: `def add(a: int, b: int) -> int`
- Why untyped Python becomes unreadable after 200 lines
- `dataclasses` — instead of dictionaries for structured data
- Default arguments, `*args`, `**kwargs` — actually understanding them

Sunday session:

- Exception handling properly: `try/except/finally`, specific exceptions vs bare `except`
- Custom exceptions: when and why to define your own
- Real example: a file reader that handles missing file, wrong format, empty file — all separately
- Difference between an error you handle and an error you let crash (both are valid choices)

Assignment: Write a student grade calculator with proper type hints, custom exceptions for invalid input, and clean error messages. No bare `except`. Submit to GitHub.

Week 3 — File I/O: Reading and Writing Everything

Saturday session:

- Text files: `open()`, `read()`, `readlines()`, `write()`, context managers (`with` statement — why it exists)
- CSV files: `csv` module, reading rows, writing rows, handling headers
- Real use case: reading a CSV of student marks, calculating averages, writing results to a new CSV

Sunday session:

- JSON files: `json.load()`, `json.dump()`, pretty printing, handling nested structures
- When to use JSON vs CSV vs plain text
- `.env` files: what they are, `python-dotenv` library, never hardcode secrets
- `pathlib` module: the modern way to handle file paths (works on Windows and Linux without breaking)

Assignment: Build a contact book — reads contacts from a JSON file, allows adding/searching/deleting contacts, saves back to JSON on exit. Uses `pathlib` for all file paths. Configuration (file path) loaded from `.env`.

Week 4 — Git & GitHub: Engineering Collaboration

Saturday session:

- What Git actually is — not "cloud backup", it's a history of decisions
- `git init`, `git add`, `git commit` — with meaningful commit messages (not "fixed stuff")
- `.gitignore` — what to never commit: `.env`, `__pycache__`, `venv/`, large data files
- `git log`, `git diff`, `git status` — reading your own history

Sunday session:

- GitHub: creating a repo, connecting local to remote, `git push`, `git pull`
- Branching: `git branch`, `git checkout`, `git merge` — why you never work on `main` directly
- Pull Requests: creating one, reviewing one, merging
- Collaborative workflow: fork → branch → PR → merge

Assignment: Take the Week 3 contact book, create a proper GitHub repo with a descriptive README, a `.gitignore`, and push it. Then make one improvement on a separate branch and merge it via a PR. This is how all future assignments will be submitted.

Month 2 — Code Quality, Modules & SQL Foundations

Week 5 — Clean Code & Project Architecture

Saturday session:

- What "clean code" actually means in practice — not aesthetic, it's about maintainability
- Single responsibility principle: one function does one thing
- Naming conventions: variables, functions, classes, files — PEP8 not optional
- Refactoring exercise: take intentionally bad code, fix it together live

Sunday session:

- Modules and packages: `__init__.py`, importing from submodules, relative vs absolute imports
- Standard project structure:

```
project/
├── src/
│   ├── __init__.py
│   ├── models.py
│   ├── utils.py
│   └── config.py
├── tests/
├── data/
├── .env
├── requirements.txt
└── README.md
```

- `config.py` pattern: centralize all configuration, never scatter magic strings

Assignment: Restructure any previous project into this folder structure. Split into at least 3 modules. Submit the refactored version as a new branch on GitHub.

Week 6 — Object Oriented Programming for Engineers

Saturday session:

- Classes and objects — not the abstract definition, a real example: a `DataLoader` class that reads different file types
- `__init__`, instance variables, methods
- Why OOP: when you have 5 functions that all share the same data, make a class

Sunday session:

- Inheritance: a `BaseModel` class that `TextClassifier` and `ImageClassifier` both inherit from
- `@property`, `@staticmethod`, `@classmethod` — what each is for

- Dunder methods: `__repr__`, `__str__`, `__len__` — making your classes behave like built-in Python objects
- Composition vs inheritance — composition is usually better

Assignment: Build a `Library` system with `Book`, `Member`, and `Library` classes. Members can borrow/return books. Library tracks availability. Proper OOP structure, pushed to GitHub.

Week 7 — SQL & SQLite: Databases From First Principles

Saturday session:

- What a database is and why files aren't enough — the contact book from Week 3 breaks at 10,000 contacts
- Relational model: tables, rows, columns, primary keys, foreign keys
- SQLite: file-based, no server needed, perfect for learning
- Core SQL: `CREATE TABLE`, `INSERT`, `SELECT`, `WHERE`, `UPDATE`, `DELETE`

Sunday session:

- Joins: `INNER JOIN`, `LEFT JOIN` — with real examples (students table + grades table)
- Aggregations: `COUNT`, `SUM`, `AVG`, `GROUP BY`, `HAVING`
- Indexes: what they are, when to add one, why `SELECT` on an unindexed column on 1M rows is slow
- Python + SQLite: `sqlite3` module, executing queries from Python, fetching results

Assignment: Migrate the contact book from JSON to SQLite. Support add/search/delete/list operations using SQL queries. All database logic in a separate `database.py` module.

Week 8 — PostgreSQL & SQLAlchemy: Production Databases

Saturday session:

- Why SQLite isn't for production: single writer, no network access
- PostgreSQL: installing locally, `psql` CLI basics, creating a database and user
- Same SQL you know, now on a real database server
- Connecting Python to PostgreSQL: `psycopg2`

Sunday session:

- SQLAlchemy: why raw SQL strings in Python code become a maintenance problem
- ORM basics: defining models as Python classes, `Session`, `query()`
- `alembic` for migrations: never manually `ALTER TABLE` in production

- Real pattern: `models.py` defines tables, `database.py` handles connection, `crud.py` handles operations

Assignment: Rebuild the contact book one more time — now on PostgreSQL with SQLAlchemy ORM. Three files: `models.py`, `database.py`, `crud.py`. This is the architecture used in every professional Python backend.

Phase 0 Capstone — Weeks 9–10 (bonus sessions or extended Sunday sessions):

Build a **Student Record Management System**:

- PostgreSQL database with at minimum: students, courses, enrollments tables
- Python CLI interface with full CRUD
- File I/O: export any table to CSV on demand
- Clean project structure, type hints throughout, custom exceptions, `.env` for DB credentials
- Full README on GitHub explaining how to run it

This capstone tests everything from Phase 0. If a student can build this cleanly, they're ready for Phase 1.

Phase 1 — Data Engineering & Visualization

Duration: 2 months

Month 3 — NumPy, Pandas & File Formats

Week 9 — NumPy: How Computers Actually Handle Numbers

Saturday session:

- Why Python lists are too slow for data — the memory layout problem
- NumPy arrays: creation, shape, dtype, memory efficiency
- Array operations: addition, multiplication — vectorized, no loops
- Indexing and slicing: 1D, 2D, 3D arrays — think of 2D as a table, 3D as a stack of tables

Sunday session:

- Broadcasting: adding a (3,) array to a (3,3) array — how and why it works
- Universal functions: `np.mean`, `np.std`, `np.sum`, `np.max`, `np.argmax`
- Boolean indexing: `arr[arr > 5]` — filter without loops
- Reshaping: `reshape`, `flatten`, `transpose` — critical for ML input preparation

- Random number generation: `np.random` — seeding for reproducibility

Assignment: Given a CSV of exam scores, use NumPy to compute mean, std, median per subject, find students below passing threshold, normalize scores to 0–1 range. No loops allowed — vectorized operations only.

Week 10 — Pandas Part 1: Data Loading & Cleaning

Saturday session:

- `DataFrame` vs `Series` — the mental model: DataFrame is a table, Series is one column
- Loading data: `pd.read_csv()`, `pd.read_json()`, `pd.read_excel()` — and their gotchas (encoding, separators, headers)
- First look at data: `.head()`, `.info()`, `.describe()`, `.shape`, `.dtypes`
- Selecting data: column selection, `.loc[]` vs `.iloc[]` — the most confused topic in Pandas

Sunday session:

- Missing data: `.isna()`, `.dropna()`, `.fillna()` — and when to use each
- Duplicates: `.duplicated()`, `.drop_duplicates()`
- Data types: converting columns, `pd.to_datetime()`, `pd.to_numeric()`
- String operations: `.str.lower()`, `.str.strip()`, `.str.contains()` — cleaning messy text columns

Assignment: Given a deliberately messy CSV (missing values, wrong dtypes, duplicate rows, inconsistent strings), clean it completely using Pandas. Document every cleaning decision in comments.

Week 11 — Pandas Part 2: Transformation & Analysis

Saturday session:

- Filtering rows: boolean conditions, multiple conditions with `&` and `|`
- Adding/modifying columns: arithmetic, `apply()` with a function, `map()`
- Groupby: `.groupby()` + `.agg()` — compute statistics per group
- Sorting: `.sort_values()`, multi-column sort

Sunday session:

- Merging DataFrames: `pd.merge()` — equivalent to SQL JOIN, with the same join types
- Concatenating: `pd.concat()` — stacking DataFrames vertically or horizontally

- Pivot tables: `.pivot_table()` — restructuring data for reporting
- `apply()` vs vectorized operations — when each is appropriate, performance difference

Assignment: Given two CSV files — one with student info, one with their project scores — merge them, compute rankings per category, produce a summary table grouped by department. Export final result to both CSV and JSON.

Week 12 — Matplotlib: Visualization From First Principles

Saturday session:

- The `Figure` and `Axes` mental model — everything in Matplotlib sits inside these two objects
- `plt.subplots()` — the right way, not `plt.plot()` directly
- Line plots, scatter plots, bar charts — with real data from your Pandas assignments
- Labels, titles, legends, axis limits — the minimum for a readable plot

Sunday session:

- Subplots: multiple plots in one figure, sharing axes
- Customization: colors, markers, line styles, font sizes
- Saving figures: `savefig()` with DPI control — for reports and papers
- Plotting directly from Pandas: `df.plot()` — convenience vs control tradeoff
- Common mistakes: overlapping labels, missing units, unreadable legends — and how to fix them

Assignment: Take the Week 11 analysis output and create a 4-panel figure: distribution of scores (histogram), scores by department (bar chart), score correlation (scatter), trend over time (line). Save as high-resolution PNG. This goes in their GitHub portfolio.

Month 4 — Statistics, Statistical Tests & Seaborn

Week 13 — Descriptive Statistics & Probability

Saturday session:

- Why statistics matters for AI — your model's evaluation metrics are statistics
- Measures of central tendency: mean, median, mode — when each is misleading
- Spread: variance, standard deviation, IQR — standard deviation is not always the right choice
- Distributions: normal, uniform, Bernoulli, Poisson — with real examples for each (not just textbook)

- Skewness and kurtosis — what they tell you about your data before you model it

Sunday session:

- Probability basics: events, probability rules, conditional probability
- Bayes' theorem — with an example from spam detection, not coin flips
- Random variables: discrete vs continuous
- Central Limit Theorem: why it matters — the reason most ML math assumes normality
- Hands-on with `scipy.stats`: computing distributions, PDFs, CDFs

Assignment: Given a real dataset, compute full descriptive statistics, identify the distribution type, check for skewness, and write a 1-page data profile report (in code + comments, not a Word doc).

Week 14 — Inferential Statistics & Hypothesis Testing

Saturday session:

- The core question: is this difference real or just noise?
- Null hypothesis, alternative hypothesis — framing the question correctly
- p-value: what it actually means and what it does not mean (this is widely misunderstood)
- Type I and Type II errors — false positive vs false negative in real terms
- Statistical significance vs practical significance — a difference can be significant and useless

Sunday session:

- t-test: one-sample, independent two-sample, paired — when to use each
- Chi-square test: for categorical data — does this distribution match expectations
- ANOVA: comparing means across more than two groups
- Mann-Whitney U: when your data is not normally distributed
- All implemented in `scipy.stats` with real data — not just formula derivation

Assignment: Given two groups of model predictions (e.g., two versions of a classifier), use the appropriate statistical test to determine if the performance difference is statistically significant. Write up the hypothesis, test choice, result, and interpretation.

Week 15 — Correlation, Regression & Statistical Thinking in AI

Saturday session:

- Correlation: Pearson, Spearman, Kendall — which to use and when

- Correlation is not causation — with an AI-specific example (feature correlation vs causal features)
- Covariance matrix — the foundation of PCA and many ML algorithms
- Scatter matrix: visualizing pairwise relationships in a dataset

Sunday session:

- Linear regression from scratch with `numpy` — understand the math before using sklearn
- Ordinary Least Squares: what it minimizes, why it works
- R-squared: what it measures, why it can be misleading
- Residual analysis: checking regression assumptions
- `scipy.stats.linregress` and `sklearn.linear_model.LinearRegression` — same math, different interfaces

Assignment: Given a dataset with multiple features, find the feature most correlated with the target, run a linear regression, evaluate with R-squared and residual plot, check correlation assumptions. Full analysis notebook pushed to GitHub.

Week 16 — Seaborn & Publication-Quality Visualization

Saturday session:

- Seaborn's relationship to Matplotlib: built on top of it, but dataset-aware
- Distribution plots: `histplot`, `kdeplot`, `ecdfplot` — understanding your data's shape
- Categorical plots: `boxplot`, `violinplot`, `stripplot`, `swarmplot`
- When to use each: boxplot hides distribution shape, violin shows it — that choice matters

Sunday session:

- Relationship plots: `scatterplot`, `lineplot`, `regplot` with confidence intervals
- `heatmap` for correlation matrices — the standard way to show feature relationships
- `pairplot` for exploratory data analysis — overview of an entire dataset in one call
- `FacetGrid` for small multiples — same plot across different groups
- Publication-quality figures: themes (`set_theme`), palettes, figure sizing for papers and reports

Assignment: Take the Week 11 dataset and produce a full exploratory data analysis (EDA) report using Seaborn — at minimum 6 different plot types, a correlation heatmap, and a pairplot. This is the EDA they'll run on every dataset going forward.

Phase 1 Capstone:

Full data analysis project on a real-world dataset (Kaggle or your own):

- Load and clean with Pandas
- Full descriptive statistics
- At least two statistical tests with written interpretation
- Complete visualization suite (Matplotlib + Seaborn)
- PostgreSQL: store cleaned data, run aggregation queries
- Final report as a Jupyter notebook with markdown cells explaining every decision
- Pushed to GitHub with a proper README

Phase 2 — Machine Learning Engineering

Duration: 2.5 months

Month 5 — Classical ML & Scikit-learn

Week 17 — The ML Engineering Mindset & Scikit-learn Architecture

Saturday session:

- What ML actually is: finding a function from data, not magic
- The three questions before any ML project: What is X? What is Y? What does "good" mean?
- Scikit-learn's unified API: `.fit()`, `.predict()`, `.transform()` — why everything follows this pattern
- Train/validation/test split — why three splits, not two: the hidden overfitting trap with validation sets
- `train_test_split`, stratification for imbalanced classes

Sunday session:

- Feature matrix X and target vector y — data preparation conventions
- Data leakage: the most common and dangerous mistake in ML — fitting a scaler on the full dataset before splitting
- `Pipeline`: why you should chain preprocessing + model together, never separately
- Baseline models: always establish a baseline before training anything — predict the majority class, predict the mean
- Evaluating a baseline: this is what "better than random" actually means

Assignment: Take the Phase 1 capstone dataset, define an ML problem on it, create proper train/val/test splits, build a baseline model, evaluate it. No actual ML model yet — just setup and baseline. Submit with full code and comments.

Week 18 — Supervised Learning: Classification

Saturday session:

- Logistic Regression: not for regression despite the name — probability output, decision boundary
- Decision Trees: intuitive, interpretable, but overfit easily
- Random Forest: why averaging many bad trees gives a good result — the ensemble intuition
- Hands-on: train all three on the same dataset, compare results

Sunday session:

- Classification metrics properly: accuracy is almost always the wrong metric
- Precision, Recall, F1 — with a medical diagnosis analogy: you'd rather have false positives than miss cancer
- Confusion matrix: reading it, what each cell means
- ROC curve and AUC: threshold-independent evaluation
- `classification_report` in sklearn: reading it properly

Assignment: Build a binary classifier on a real dataset (e.g., email spam, loan default). Train 3 models, evaluate with F1 and AUC, produce a confusion matrix for each. Write a comparison: which model would you deploy and why?

Week 19 — Supervised Learning: Regression & Feature Engineering

Saturday session:

- Linear Regression in sklearn: same math from Phase 1, now with pipelines
- Ridge and Lasso: regularization — why adding a penalty to the loss prevents overfitting
- Polynomial features: when the relationship isn't linear
- Regression metrics: MAE, MSE, RMSE, R-squared — what each penalizes differently

Sunday session:

- Feature engineering: creating new features from existing ones — often more impactful than better models
- Encoding categorical variables: `OneHotEncoder`, `OrdinalEncoder` — which for which
- Feature scaling: `StandardScaler`, `MinMaxScaler`, `RobustScaler` — when each is appropriate
- Feature selection: `SelectKBest`, correlation-based filtering, feature importance from tree models

Assignment: House price prediction. Engineer at least 3 new features, handle categorical variables properly, build Ridge and Lasso models, tune regularization strength, evaluate on held-out test set. Compare with unengineered baseline.

Week 20 — Model Evaluation, Cross-Validation & Hyperparameter Tuning

Saturday session:

- Cross-validation: why a single train/val split gives noisy estimates
- K-Fold, Stratified K-Fold — the right default for most problems
- `cross_val_score` — running CV properly in sklearn
- Learning curves: diagnosing underfitting vs overfitting visually

Sunday session:

- Hyperparameter tuning: `GridSearchCV`, `RandomizedSearchCV`
- The nested CV trap: never tune hyperparameters on the same fold you evaluate on
- Early stopping concept: for iterative models, when to stop training
- Putting it together: a full ML pipeline with preprocessing, model, CV evaluation, and hyperparameter tuning as one `Pipeline` object

Assignment: Take any previous classification model and implement full K-Fold CV + GridSearchCV for hyperparameter tuning. Plot learning curves. Report final test performance — evaluated only once, at the very end.

Month 6 — Unsupervised Learning, Advanced ML & Experiment Tracking

Week 21 — Unsupervised Learning: Clustering & Dimensionality Reduction

Saturday session:

- K-Means: the algorithm, the objective function, the elbow method for choosing K
- When clustering is useful: customer segmentation, data exploration, preprocessing for semi-supervised learning
- DBSCAN: density-based clustering — doesn't require specifying K, handles noise
- Cluster evaluation: silhouette score, inertia — evaluating without labels

Sunday session:

- PCA: reducing dimensions while preserving variance — the linear algebra intuition without full derivation

- When to use PCA: visualization (reduce to 2D), preprocessing before ML, removing correlated features
- t-SNE: for visualization only — not for preprocessing
- UMAP: faster than t-SNE, better for large datasets — increasingly standard in NLP and bioinformatics
- Visualizing high-dimensional data: embedding your RUEmoCorp dataset to 2D as a live demo

Assignment: Take a dataset with 10+ features, run K-Means clustering, evaluate with silhouette score, visualize clusters with PCA 2D projection using Seaborn. Identify what each cluster represents in real terms.

Week 22 — Imbalanced Data, Pipelines & Production-Ready ML Code

Saturday session:

- Class imbalance: why it breaks standard metrics and models — fraud detection example
- Oversampling: SMOTE — generate synthetic minority samples, not just duplicate
- Undersampling: when you have enough majority class data to throw away
- Class weights: the simplest solution — `class_weight='balanced'` in sklearn

Sunday session:

- Full sklearn `Pipeline` with `ColumnTransformer`: handle numeric and categorical columns differently in one pipeline
- `joblib`: saving and loading models — a model that can't be saved is a model that can't be deployed
- Model versioning: naming convention for saved models, when to retrain
- Writing an inference function: takes raw input, applies the same pipeline, returns a prediction — this is what an API will call

Assignment: Build a complete fraud detection pipeline: handle severe class imbalance, full ColumnTransformer for mixed data types, save with joblib, write an `predict(raw_input: dict) -> dict` function that can be imported and called by a future API.

Week 23 — MLflow: Experiment Tracking & Model Registry

Saturday session:

- The problem MLflow solves: "which hyperparameters gave that good result last Tuesday?"
- MLflow tracking: runs, experiments, logging parameters, metrics, artifacts

- Integrating MLflow into a training script — 10 lines of code that save everything
- MLflow UI: reading your experiment history visually

Sunday session:

- MLflow model registry: promoting a model from "experiment" to "staging" to "production"
- Model signatures: defining input/output schema so the model is self-documenting
- Comparing runs: selecting the best model across 20 experiments in 2 clicks
- Connecting MLflow to a remote tracking server (even a simple SQLite backend on a shared machine)

Assignment: Take the Week 22 fraud detection pipeline, wrap the entire training script with MLflow logging. Run 5 experiments with different hyperparameters. Use the MLflow UI to identify the best run. Register that model. Screenshot the UI, push everything to GitHub.

Phase 2 Capstone:

End-to-end ML project:

- Real dataset (student chooses, you approve)
- Full EDA with Seaborn
- Feature engineering
- At least 3 models compared
- Full CV evaluation
- Hyperparameter tuning
- MLflow experiment tracking
- Final model saved with joblib
- `predict()` function ready for API integration
- GitHub repo with clean structure, README, and reproducible `requirements.txt`

Phase 3 — Deep Learning & NLP

Duration: 2.5 months

Month 7 — Deep Learning Foundations

Week 24 — Neural Networks From Scratch

Saturday session:

- The perceptron: input → weighted sum → activation → output

- Why deep learning: representation learning — the network learns features, you don't engineer them
- Forward pass: matrix multiplication through layers — connect this to NumPy from Phase 1
- Activation functions: ReLU (most common), sigmoid (outputs), softmax (multiclass) — not a zoo of options, practical selection rules

Sunday session:

- Loss functions: cross-entropy for classification, MSE for regression — why each
- Backpropagation: the chain rule applied repeatedly — understand the concept, not every derivative
- Gradient descent: how the network learns — visualize the loss surface
- Build a 2-layer neural network from scratch with NumPy: forward pass, loss, backward pass, weight update. No PyTorch yet.

Assignment: Implement a neural network from scratch in NumPy that classifies XOR (which linear models can't). No PyTorch, no sklearn. Pure math in code.

Week 25 — PyTorch: The Engineering Way

Saturday session:

- Why PyTorch over NumPy: autograd — automatic differentiation, no manual backprop
- Tensors: same as NumPy arrays but on GPU-capable
- `requires_grad`, `.backward()`, `.grad` — PyTorch's computation graph
- Reimplement Week 24's network in PyTorch in 20 lines

Sunday session:

- `nn.Module`: the base class for every neural network in PyTorch — `__init__` and `forward`
- `nn.Linear`, `nn.ReLU`, `nn.Sequential` — building blocks
- Training loop structure: forward pass → loss → `optimizer.zero_grad()` → `loss.backward()` → `optimizer.step()`
- `DataLoader` and `Dataset`: loading data efficiently for training, batching, shuffling

Assignment: Build an image classifier on MNIST (handwritten digits) using PyTorch. A proper `Dataset` class, `DataLoader`, training loop with loss logging, validation accuracy after each epoch. Achieves >95% accuracy.

Week 26 — CNNs & Computer Vision Basics

Saturday session:

- Why fully connected networks fail at images: too many parameters, no spatial awareness
- Convolution: sliding a filter over an image — what it detects at each layer
- Pooling: downsampling — reduce spatial dimensions, keep important features
- CNN architecture: Conv → ReLU → Pool → repeat → Flatten → Linear

Sunday session:

- Transfer learning: ImageNet-pretrained models have already learned edges, textures, shapes — you fine-tune the last layers
- `torchvision.models`: ResNet, EfficientNet — loading pretrained weights
- Fine-tuning vs feature extraction: when to freeze layers, when to train them all
- Real project: classify a custom image dataset using ResNet fine-tuning

Assignment: Build an image classifier for a 3-class custom dataset (they collect ~100 images each of 3 objects using their phone). Fine-tune ResNet18. Evaluate with F1. Deploy as a function that takes an image path and returns predicted class.

Week 27 — Recurrent Networks & Sequence Modeling

Saturday session:

- Why CNNs fail at sequences: order matters in text and audio
- RNNs: processing sequences step by step, hidden state as memory
- The vanishing gradient problem: why RNNs forget long-range dependencies
- LSTMs: the solution — gates that control what to remember and forget (conceptual, not full math)

Sunday session:

- GRUs: lighter than LSTM, often same performance
- Sequence classification with PyTorch: text sentiment with an LSTM
- Padding and packing variable-length sequences: `pad_sequence`, `pack_padded_sequence`
- When RNNs are still relevant: time series, audio features — not dead despite Transformers

Assignment: Build a sequence classifier (SMS spam detection) using an LSTM in PyTorch. Proper tokenization, padding, training loop, evaluation.

Month 8 — NLP Engineering

Week 28 — Text Processing & Classical NLP

Saturday session:

- Text preprocessing pipeline: lowercasing, punctuation removal, tokenization, stopword removal, stemming vs lemmatization
- Bag of Words: `CountVectorizer` — term frequency representation
- TF-IDF: `TfidfVectorizer` — downweighting common words, upweighting rare ones
- Text classification with TF-IDF + Logistic Regression: still a strong baseline

Sunday session:

- Word embeddings: Word2Vec — words as dense vectors, similar words cluster together
- GloVe: pretrained embeddings you can download and use immediately
- Why embeddings beat BoW: "king" - "man" + "woman" = "queen"
- Loading pretrained Word2Vec/GloVe with `gensim`, using as input to an LSTM

Assignment: Build a text classifier three ways: BoW + Logistic Regression, TF-IDF + Random Forest, GloVe + LSTM. Compare performance. Which wins and why?

Week 29 — Transformers: Architecture & Intuition

Saturday session:

- The attention mechanism: instead of reading left-to-right, look at all words simultaneously
- Self-attention: every word attends to every other word — the relevance score
- Multi-head attention: multiple attention patterns in parallel
- Positional encoding: how Transformers know word order without recurrence

Sunday session:

- The full Transformer: encoder, decoder, encoder-decoder
- BERT architecture: encoder-only, masked language modeling pretraining
- GPT architecture: decoder-only, causal language modeling
- Why fine-tuning works: pretraining on massive data learns general language, fine-tuning adapts to your task

Assignment: No coding this week — write a 2-page technical explanation of how attention works, with diagrams drawn by hand or in code. Understanding before usage.

Week 30 — HuggingFace: Transformers in Practice

Saturday session:

- HuggingFace ecosystem: `transformers`, `datasets`, `evaluate`, `hub`
- `AutoTokenizer`, `AutoModel`, `AutoModelForSequenceClassification` — the Auto classes
- Tokenization properly: subword tokenization, `input_ids`, `attention_mask`, `token_type_ids`
- Running inference: loading a pretrained model and making predictions

Sunday session:

- Fine-tuning with `Trainer` API: `TrainingArguments`, `Trainer`, `compute_metrics`
- Fine-tuning XLM-RoBERTa on a text classification task — this is directly your Roman Urdu work
- Pushing a fine-tuned model to HuggingFace Hub: model card, README, usage example
- Evaluating properly: macro F1, per-class metrics for multilabel scenarios

Assignment: Fine-tune `xlm-roberta-base` on a multilingual sentiment dataset. Push the model to HuggingFace Hub with a proper model card. This is the same pipeline you used for emotion recognition — students are reproducing real published work.

Week 31 — Speech AI Engineering

Saturday session:

- Audio representation: waveform, sampling rate, Fourier transform, spectrogram, mel-spectrogram
- Why mel-spectrograms: human hearing is logarithmic — mel scale matches perception
- `librosa`: the standard Python library for audio — loading, resampling, feature extraction
- MFCC features: compact audio representation used in classical speech systems

Sunday session:

- OpenAI Whisper: automatic speech recognition — loading, transcribing, handling different languages
- Speech-to-text pipeline: audio file → Whisper → text → downstream NLP
- Your vocal fatigue work as a live case study: what features you extracted, the pipeline, the deployment
- Text-to-speech: `TTS` library basics — generating speech from text

Assignment: Build a speech processing pipeline: record or download audio, transcribe with Whisper, run sentiment analysis on the transcript with a HuggingFace model, output structured JSON. End-to-end audio → text → analysis.

Phase 3 Capstone:

Build a complete NLP system of their choice (approved by you):

- Options: emotion classifier, intent detector, multilingual QA, document summarizer
- Must use a fine-tuned Transformer model
- Must push model to HuggingFace Hub with proper model card
- Must include proper evaluation (not just accuracy)
- Must have a runnable inference script
- Pushed to GitHub with full documentation

Phase 4 — AI Systems & LLM Engineering

Duration: 2 months

Month 9 — APIs, LLM Engineering & RAG

Week 32 — Building Production AI APIs with FastAPI

Saturday session:

- FastAPI recap but deeper: request validation with Pydantic models, response models
- Async endpoints: `async def` — why it matters for AI APIs that call external services
- Background tasks: running inference asynchronously
- API versioning: `/v1/predict` not `/predict` — production APIs have versions

Sunday session:

- Loading ML models at startup, not per request: `lifespan` context manager
- Rate limiting: preventing abuse
- API authentication: API keys with `Header` dependency
- Structuring an AI API: `routers/`, `models/`, `services/`, `core/` — each with a single responsibility

Assignment: Wrap the Phase 3 capstone model in a FastAPI application. Proper Pydantic validation, API key authentication, model loaded once at startup, async endpoint, structured JSON response.

Week 33 — LLM Engineering: OpenAI & Anthropic APIs

Saturday session:

- LLM API basics: system prompt, user message, assistant response — the conversation structure
- Token management: counting tokens, staying within context limits, `tiktoken`
- Temperature and sampling parameters: what they control, when to use deterministic (temperature=0) vs creative settings
- Structured outputs: getting JSON back reliably — function calling / tool use

Sunday session:

- Prompt engineering with discipline: chain-of-thought, few-shot examples, role prompting
- Common failure modes: hallucination, instruction following failures, context window overflow
- Streaming responses: `stream=True` for real-time output in applications
- Cost management: estimating cost before running, caching repeated prompts

Assignment: Build a document analysis API that accepts a PDF or text, uses an LLM to extract structured information (entities, summary, key points), returns validated JSON. Error handling for malformed LLM output.

Week 34 — Vector Databases & RAG Systems

Saturday session:

- The problem RAG solves: LLMs have a knowledge cutoff and can't reason over your private data
- Embeddings as search: converting text to vectors so semantically similar text is numerically close
- Vector databases: ChromaDB (local, great for learning), Qdrant (production-grade)
- Building a basic retriever: embed documents, store in ChromaDB, query with a question

Sunday session:

- Full RAG pipeline: document ingestion → chunking → embedding → storage → retrieval → LLM generation
- Chunking strategies: fixed-size, sentence-based, semantic — how chunk size affects retrieval quality
- Retrieval quality: relevance vs diversity, top-k selection
- Evaluation: does the retrieved chunk actually answer the question? RAGAS metrics conceptually

Assignment: Build a RAG system over a real document set (your lab curriculum docs, a textbook chapter, research papers). Accept a natural language question, retrieve relevant chunks, generate an answer with citations. FastAPI endpoint.

Week 35 — AI Agents & Tool Use

Saturday session:

- What agents actually are: LLM + ability to call functions + loop until task is complete
- Function calling: defining tools the LLM can invoke, parsing tool call responses
- The ReAct pattern: Reason → Act → Observe → repeat
- Building a simple agent: web search tool + calculator tool + the LLM orchestrating them

Sunday session:

- Agent failure modes: infinite loops, wrong tool selection, hallucinated tool parameters
- Human-in-the-loop: when to pause and ask for confirmation
- Memory for agents: conversation history, tool call history, summarization to stay within context
- LangChain vs direct API: LangChain is convenient but hides what's happening — understand direct API first

Assignment: Build an agent that can answer questions about a topic by searching the web (use a search API), reading a URL, and synthesizing an answer. Must handle cases where search returns irrelevant results.

Phase 4 Capstone:

Build a complete AI application:

- A domain-specific assistant (e.g., a lab FAQ bot trained on your curriculum, a medical report summarizer, a code reviewer)
- RAG backend for knowledge retrieval
- LLM for generation
- FastAPI serving it as an API
- Simple frontend (Streamlit is fine at this stage)
- Deployed to a free cloud tier
- GitHub with full README and architecture diagram

Phase 5 — MLOps & Deployment

Duration: 1.5 months

Month 11 — Docker, CI/CD & Cloud Deployment

Week 36 — Docker for AI Systems

Saturday session:

- Docker mental model: not a VM, it's a reproducible process environment
- `Dockerfile`: FROM, RUN, COPY, WORKDIR, ENV, CMD — building an image for an AI API
- Multi-stage builds: separate build environment from runtime environment — smaller final images
- `.dockerignore`: don't copy your entire dataset into the image

Sunday session:

- `docker-compose`: running multi-container setups — API + PostgreSQL + Redis together
- Volume mounts: persisting model files and database data outside the container
- Environment variables in Docker: never hardcode secrets in Dockerfile
- Debugging containers: `docker logs`, `docker exec -it`, common failure patterns

Assignment: Containerize the Phase 4 capstone. Write a `Dockerfile` and `docker-compose.yml` that spins up the API and a PostgreSQL database together. It should run with one command: `docker-compose up`.

Week 37 — GitHub Actions: CI/CD for AI Projects

Saturday session:

- What CI/CD means: every push triggers automated testing and optionally deployment
- GitHub Actions: triggers, jobs, steps, runners
- Writing a workflow: on push to main → install dependencies → run tests → report pass/fail
- `pytest` integration: if tests fail, the deployment stops

Sunday session:

- Building a Docker image in CI: build and push to Docker Hub on every merge to main
- Environment secrets in GitHub Actions: API keys, database URLs — never in the YAML file
- Deployment trigger: after build succeeds, deploy to cloud
- Notification on failure: email or Slack alert when a build breaks

Assignment: Add a complete CI/CD pipeline to the Phase 4 capstone: run tests on every push, build Docker image on merge to main, push to Docker Hub.

Week 38 — Cloud Deployment & Monitoring

Saturday session:

- Cloud options in budget: Render, Railway, Fly.io (all have free tiers)
- Deploying a Dockerized AI API: environment variables, persistent storage, custom domains
- Health checks: `/health` endpoint — how cloud platforms verify your service is alive
- Scaling concepts: horizontal scaling, load balancing — conceptual at this stage

Sunday session:

- Logging properly in production: structured logs (JSON), log levels, request ID tracing
- Monitoring basics: what to measure — latency, error rate, request volume
- Prometheus + Grafana: instrumenting a FastAPI app, building a basic dashboard
- Data drift concept: your model was trained on yesterday's data, but production data changes

Assignment: Deploy the fully containerized Phase 4 capstone to Render or Railway. Add a `/health` endpoint, structured logging, and at minimum a Prometheus metrics endpoint. Share the live URL.

Phase 5 Capstone — Final Portfolio Project:

Take one project from any previous phase and make it fully production-grade:

- Containerized with Docker
- CI/CD pipeline on GitHub Actions
- Deployed to cloud with a live URL
- Monitoring endpoint
- Structured logging
- Architecture diagram
- Full README with: problem statement, approach, how to run locally, how to deploy, API documentation
- This is the project students show in every job interview and every portfolio review

---

Assessment & Progression Gates

Between each phase, a student must pass a **gate assessment** before moving to the next phase.

Gate	Expected Outcomes per phase
Phase 0 → 1	Build and push a clean Python project with SQL backend to GitHub in 2 hours
Phase 1 → 2	Given a messy dataset, produce a full EDA with statistical tests and visualizations
Phase 2 → 3	Build, evaluate, and save a complete sklearn ML pipeline with MLflow tracking
Phase 3 → 4	Fine-tune a HuggingFace model, push to Hub, write inference function
Phase 4 → 5	Build and deploy a functional AI API with authentication and structured responses
Phase 5 → Complete	Live deployed project, CI/CD working, monitoring active, architecture documented

---